

CROP DISEASE DETECTION AND CLASSIFICATION FOR FARMERS

PROJECT REPORT

Submitted by

HARISH R P 230701106

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2026

CROP DISEASE DETECTION AND CLASSIFICATION FOR FARMERS

A Mini Project Report

Submitted by

HARISH R P

Department of Computer Science and Engineering

Academic Year 2025 – 2026

May 2026

BONAFIDE CERTIFICATE

Certified that this project report titled crop disease detection and classification for farmers is the bonafide work carried out by HARISH R P under the supervision and guidance of the faculty in charge in partial fulfillment of the requirements for the Mini Project in the Department of Computer Science and Engineering during the academic year 2025 – 2026.

**SIGNATURE OF THE FACULTY INCHARGE SIGNATURE OF THE INTERNAL
EXAMINER**

Submitted for the Practical Examination held on _____

TABLE OF CONTENTS

1. INTRODUCTION

- 1.1 Background and Motivation
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Report

2. LITERATURE REVIEW

- 2.1 Traditional Crop Disease Detection Methods
- 2.2 AI-Based Plant Disease Detection Systems
- 2.3 Existing Smart Farming Applications
- 2.4 Gap Identified

3. SYSTEM DESCRIPTION

- 3.1 Project Overview
- 3.2 Features of the System
- 3.3 Technology Stack
- 3.4 System Architecture

4. METHODOLOGY

- 4.1 Image Capture and Upload
- 4.2 Image Validation
- 4.3 Excess Green Index (ExG) Processing
- 4.4 Gemini AI Disease Analysis
- 4.5 Diagnosis Generation
- 4.6 User Interface Workflow
- 4.7 Error Handling and Fallback Support

5. IMPLEMENTATION DETAILS

- 5.1 Frontend Design
- 5.2 AI Integration
- 5.3 Camera Handling Mechanism
- 5.4 Computer Vision Utilities

5.5 Output Generation and Display

6. RESULTS AND ANALYSIS

6.1 Functional Results

6.2 AI Diagnosis Evaluation

6.3 User Interface Evaluation

7. DISCUSSION

8. CONCLUSION AND FUTURE WORK

9. REFERENCES

10. APPENDIX A — SOURCE CODE

11. APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

ABSTRACT

Agriculture plays a vital role in food production and economic development. Plant diseases significantly affect crop productivity and quality, leading to financial losses for farmers. Early identification of plant diseases is essential to prevent the spread of infections and improve agricultural yield. Traditional disease diagnosis methods require expert knowledge and physical inspection, which may not always be accessible to farmers in remote regions.

This project presents AgroPulse — AI Crop Doctor, a browser-based intelligent crop disease diagnosis application developed using React, TypeScript, Vite, Tailwind CSS, and Google Gemini AI. The system allows users to capture leaf images using a live browser camera or upload images from the device gallery. The uploaded leaf image is processed using computer vision techniques, including the Excess Green Index (ExG), to analyze vegetation health and improve disease visibility.

The processed image is then analyzed using Google Gemini AI, which generates disease diagnosis results along with possible symptoms and recommendations. The application provides a modern and responsive user interface that enables users to interact with the system easily without installing additional software.

The project demonstrates the practical integration of artificial intelligence, computer vision, and modern web technologies in agriculture. AgroPulse serves as an educational and real-world demonstration of AI-assisted smart farming systems. Future enhancements may include multilingual support, real-time farmer assistance, offline AI inference, and integration with IoT-based agricultural monitoring systems.

1. INTRODUCTION

1.1 Background and Motivation

Agriculture is one of the most important sectors contributing to food security and economic growth. Plant diseases caused by bacteria, fungi, viruses, and environmental conditions reduce crop productivity and directly impact farmers' income. Early disease detection can help farmers take preventive measures before the disease spreads to large areas.

Traditional plant disease identification methods require agricultural experts and laboratory testing. These methods are time-consuming and may not be accessible in rural areas. With the advancement of artificial intelligence and computer vision, automated disease detection systems can assist farmers by providing faster and more accurate diagnosis.

This project is motivated by the need to develop a lightweight, browser-based AI crop diagnosis system that farmers and students can use easily. AgroPulse combines image processing techniques with Google Gemini AI to provide smart disease analysis using leaf images.

1.2 Problem Statement

Farmers often face difficulties in identifying crop diseases at an early stage due to lack of expert support and limited access to agricultural technologies. Existing disease detection systems may require complex installations, paid services, or specialized hardware.

There is a need for a simple and accessible web-based application that allows users to capture or upload crop leaf images and receive AI-generated disease diagnosis instantly without any installations or technical expertise.

1.3 Objectives

The objectives of this project are:

- To develop a browser-based AI crop disease diagnosis application.
- To integrate camera capture and image upload functionality.
- To implement computer vision preprocessing using Excess Green Index (ExG).
- To use Google Gemini AI for intelligent disease diagnosis.
- To provide an easy-to-use and responsive user interface for farmers and students.
- To demonstrate the integration of AI and computer vision in agriculture.

1.4 Scope of the Report

This report focuses on the design and implementation of AgroPulse using React, TypeScript, Tailwind CSS, and Google Gemini AI. The system supports image upload, browser camera access, image preprocessing, and AI-based diagnosis generation. The report includes system architecture,

implementation details, preprocessing methodology, and performance analysis. Advanced agricultural analytics, IoT integration, and offline AI models are beyond the scope of this project.

2. LITERATURE REVIEW

Plant disease detection has been an active research area with approaches ranging from manual field inspection to deep learning-based automated systems.

2.1 Traditional Crop Disease Detection Methods

Traditional crop disease detection methods involve manual inspection of leaves by agricultural experts. Farmers identify diseases based on visible symptoms such as spots, discoloration, wilting, or fungal growth. Although these methods can be accurate, they require expert knowledge and significant time. Environmental factors and human errors may also affect diagnosis quality.

2.2 AI-Based Plant Disease Detection Systems

Recent developments in artificial intelligence and computer vision have enabled automated crop disease detection systems. Deep learning and image classification models are widely used to analyze leaf patterns and identify diseases. AI-based systems provide faster analysis and can assist farmers in taking early preventive measures. Cloud-based AI services and machine learning APIs have improved the accessibility of smart farming solutions.

2.3 Existing Smart Farming Applications

Several agricultural applications provide crop monitoring and disease analysis functionalities. Some systems use convolutional neural networks (CNNs), while others use cloud AI services for image analysis. However, many existing applications require mobile installations, expensive subscriptions, or complex configurations. Some systems also lack browser-based camera support and lightweight deployment.

2.4 Gap Identified

Although AI-based agricultural applications are growing rapidly, many systems are not beginner-friendly and may not support simple browser-based operation. This project addresses these gaps by developing a lightweight React-based web application that integrates camera capture, image preprocessing, and Gemini AI diagnosis in a simple and interactive environment.

3. SYSTEM DESCRIPTION

3.1 Project Overview

AgroPulse is a browser-based AI crop disease diagnosis application that allows users to capture leaf images using a live camera or upload images from local storage. The system preprocesses the image using the Excess Green Index technique to highlight vegetation regions and improve analysis quality. The processed image is then analyzed using Google Gemini AI, which generates disease diagnosis and recommendations. The application provides real-time image preview, AI-generated output, and responsive interaction through a modern web interface.

3.2 Features of the System

The main features of the system include:

- Browser-based live camera capture
- Image upload functionality
- Real-time image preview
- Excess Green Index (ExG) image processing
- AI-powered crop disease diagnosis
- Google Gemini AI integration
- Responsive user interface
- Markdown-based diagnosis display
- Error handling and fallback support
- Cross-platform browser compatibility

3.3 Technology Stack

The following technologies are used in this project:

Component	Technology Used
Frontend Framework	React with TypeScript
Build Tool	Vite

Styling	Tailwind CSS
AI Integration	Google Gemini AI SDK
Image Processing	JavaScript Canvas API
Development Environment	Node.js 18+
Browser APIs	MediaDevices API
License	Apache-2.0

3.4 System Architecture

The system architecture consists of the following major source modules:

- `src/App.tsx` — main UI, camera management, image preview, and diagnosis workflow
- `src/services/geminiService.ts` — Gemini API client and prompt generation
- `src/utils/cvUtils.ts` — computer vision utilities including ExG image processing

The end-to-end workflow of the system is as follows:

1. User captures a leaf image using the browser camera or uploads one from the gallery.
2. The image is validated and displayed in the preview panel.
3. ExG preprocessing is applied to the image using the Canvas API.
4. The processed image and a diagnostic prompt are sent to the Gemini AI API.
5. Gemini AI returns a structured disease diagnosis result.

6. The diagnosis is rendered as formatted Markdown output for the user.

4. METHODOLOGY

4.1 Image Capture and Upload

The application supports both live camera capture and image upload functionality. Users can open the browser camera using the MediaDevices API and capture crop leaf images directly. If camera access is unavailable, users can upload images from their gallery using the upload feature as a fallback mechanism.

4.2 Image Validation

The system validates uploaded images before processing. Supported formats include JPG, JPEG, and PNG. Invalid files or unsupported image types are rejected with appropriate error messages to guide the user.

4.3 Excess Green Index (ExG) Processing

The Excess Green Index is a vegetation analysis technique used to highlight green regions in plant images. The ExG formula used is:

$$\text{ExG} = 2G - R - B$$

Where G represents the green channel, R represents the red channel, and B represents the blue channel of each pixel. The ExG technique amplifies the green vegetation components while suppressing non-green background regions, improving Gemini AI's ability to focus on leaf health indicators.

4.4 Gemini AI Disease Analysis

The processed image is encoded as a base64 data URI and submitted to the Google Gemini AI API along with a carefully engineered diagnostic prompt. The prompt instructs Gemini to identify visible symptoms, name the probable disease, assess severity, and suggest treatment recommendations in a structured format.

4.5 Diagnosis Generation

Gemini AI returns a Markdown-formatted response containing the disease name, symptom description, severity level, and recommended actions. The response is parsed and rendered directly in the browser using a Markdown renderer, ensuring clear and accessible presentation.

4.6 User Interface Workflow

The user interface guides the farmer through a simple three-step workflow:

1. Open the field camera or upload a leaf image from the gallery.
2. Preview the captured or uploaded image on screen.
3. Click Analyze Now to receive the AI-generated disease diagnosis.

4.7 Error Handling and Fallback Support

The application includes robust error handling at every stage. Camera permission denials are caught gracefully and users are directed to the upload option. API errors from Gemini are displayed with descriptive messages. Invalid image types trigger validation warnings before any processing is attempted.

5. IMPLEMENTATION DETAILS

5.1 Frontend Design

The frontend is built with React and TypeScript, styled using Tailwind CSS utility classes. The interface is fully responsive and works on desktop and mobile browsers. Key UI components include the camera preview panel, image upload section, analysis button, and the Markdown diagnosis output panel.

5.2 AI Integration

Google Gemini AI is integrated via the official Gemini AI SDK. The `geminiService.ts` module encapsulates all API communication. The service converts the processed canvas image to a base64 JPEG, constructs the multimodal prompt, and calls the Gemini vision model. Responses are returned as plain Markdown text for rendering.

5.3 Camera Handling Mechanism

Camera initialization is managed inside a React `useEffect` hook to ensure the video element is fully mounted before the `MediaStream` is bound to its `srcObject`. The video element is set to muted and `autoPlay` to comply with browser autoplay policies. This approach resolves the black preview issue that arises when `srcObject` is assigned before the DOM element is ready.

5.4 Computer Vision Utilities

The `cvUtils.ts` module implements the ExG algorithm using the HTML Canvas API. Each pixel's RGB values are read from the `ImageData` array, the ExG formula is applied, and the resulting enhanced image is written back to a canvas element. The canvas is then used to generate the base64 image sent to Gemini AI.

5.5 Output Generation and Display

Diagnosis output is rendered as formatted Markdown directly in the browser. The output panel displays the disease name, symptoms, severity, and recommendations in a structured, readable layout. A loading indicator is shown while the Gemini API request is in progress.

5.6 Setup and Local Deployment

Prerequisites: Node.js 18 or higher is recommended.

Step 1 — Install dependencies:

```
npm install
```

Step 2 — Create a .env file at the project root:

```
# .env
```

```
VITE_GEMINI_API_KEY=your_api_key_here
```

Step 3 — Start the development server:

```
npm run dev
```

Step 4 — Open the URL displayed by Vite in the browser (e.g., <http://localhost:3001>).

Note: The VITE_ prefix is required because Vite only exposes environment variables that begin with VITE_ to client-side code.

6. RESULTS AND ANALYSIS

6.1 Functional Results

The developed application successfully performs all implemented functions. Users can open the browser camera, capture leaf images, upload alternative images, apply ExG preprocessing, and receive AI-generated diagnoses. The system demonstrates smooth end-to-end interaction from image capture through to Markdown diagnosis output.

6.2 AI Diagnosis Evaluation

Google Gemini AI consistently produces structured diagnostic responses that identify disease name, visible symptoms, and recommended treatments. The ExG preprocessing step improves the clarity of leaf regions in images with complex backgrounds, resulting in more focused AI analysis. The integration of a well-constructed prompt template ensures consistent and useful output structure across different leaf images.

Feature	Status	Notes
Live camera capture	Functional	Requires browser camera permission
Image upload fallback	Functional	Supports JPG, JPEG, PNG
ExG image preprocessing	Functional	Canvas API implementation
Gemini AI diagnosis	Functional	Structured Markdown output
Markdown rendering	Functional	Displays disease, symptoms, treatment
Error handling	Functional	Covers camera denial and API errors
Responsive UI	Functional	Works on desktop and mobile browsers

6.3 User Interface Evaluation

The user interface is clean, responsive, and beginner-friendly. The three-step workflow (open camera, capture image, analyze) is intuitive for non-technical users. Tailwind CSS ensures a consistent visual design across screen sizes. The Markdown diagnosis panel provides well-structured, readable output that guides farmers on next steps.

7. DISCUSSION

AgroPulse successfully demonstrates the integration of modern web technologies, computer vision, and large language model AI in an agricultural application. The project's core strength lies in its accessibility: farmers with any modern browser can diagnose crop diseases without installing software or possessing technical expertise.

The use of Google Gemini AI as the diagnostic engine provides a significant advantage over traditional CNN-based classifiers. Gemini's multimodal capabilities allow it to process visual leaf features alongside a rich natural language prompt, producing detailed, contextually relevant diagnoses rather than single-label classifications.

The ExG preprocessing pipeline addresses a practical challenge in leaf disease detection: distinguishing vegetation from background noise. By amplifying green channels and suppressing others, the ExG transformation focuses Gemini's attention on the biologically relevant parts of the image.

The camera handling mechanism using `useEffect` to bind `srcObject` after DOM mounting resolves a common pitfall in browser media stream applications. The muted `autoPlay` configuration ensures compatibility across Chrome, Firefox, and Edge without triggering autoplay policy restrictions.

The primary limitation of the current system is its dependency on an active internet connection for Gemini API access. Large leaf images may also increase API response time. Future work should address these limitations through offline model inference and image compression before transmission.

8. CONCLUSION AND FUTURE WORK

This mini project presented the design and implementation of AgroPulse — AI Crop Doctor, a browser-based AI application for plant disease diagnosis. The system integrates live camera capture, Excess Green Index image preprocessing, and Google Gemini AI analysis into a seamless and accessible workflow for farmers and students.

The project demonstrates that modern web technologies — React, TypeScript, Vite, and Tailwind CSS — combined with a powerful multimodal AI API can deliver a practical agricultural tool without complex installations or backend infrastructure.

The key contributions of this project are:

- Development of a complete browser-based AI crop disease diagnosis application.
- Implementation of Excess Green Index (ExG) preprocessing using the Canvas API.
- Integration of Google Gemini AI for multimodal leaf disease analysis.
- Robust camera handling with useEffect-based srcObject binding.
- Fallback image upload support for environments without camera access.
- Responsive, beginner-friendly UI with Markdown diagnosis output.

Future work directions include:

- Multilingual support to make the application accessible to regional farmers.
- Real-time farmer assistance through a conversational AI interface.
- Offline AI inference using lightweight on-device models such as TensorFlow Lite.
- Integration with IoT-based agricultural monitoring sensors for automated alerts.
- Cloud deployment to make the application publicly accessible.
- Historical diagnosis logging so farmers can track disease progression over time.

9. REFERENCES

- [1] Google LLC, "Gemini AI API Documentation," [Online]. Available: <https://ai.google.dev/>
- [2] Meta Platforms, "React — A JavaScript Library for Building User Interfaces," [Online]. Available: <https://react.dev/>
- [3] Evan You et al., "Vite — Next Generation Frontend Tooling," [Online]. Available: <https://vite.dev/>
- [4] Tailwind Labs, "Tailwind CSS Documentation," [Online]. Available: <https://tailwindcss.com/>
- [5] Microsoft, "TypeScript Language Documentation," [Online]. Available: <https://www.typescriptlang.org/>
- [6] J. A. Gamon and J. S. Surfus, "Assessing leaf pigment content and activity with a reflectometer," *New Phytologist*, vol. 143, pp. 105–117, 1999.
- [7] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [8] Mozilla Developer Network, "MediaDevices.getUserMedia()," [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>
- [9] R. Gonzalez and R. Woods, *Digital Image Processing*, 4th ed., Pearson Education, 2018.
- [10] Apache Software Foundation, "Apache License 2.0," [Online]. Available: <https://www.apache.org/licenses/LICENSE-2.0>

APPENDIX A — SOURCE CODE

The following listings present the key source files of the AgroPulse application.

A.1 src/utlils/cvUtils.ts — ExG Computer Vision Utilities

```
// cvUtils.ts — Excess Green Index (ExG) processing

export function applyExcessGreen(imageData: ImageData): ImageData {

  const data = imageData.data;

  for (let i = 0; i < data.length; i += 4) {

    const r = data[i] / 255;

    const g = data[i + 1] / 255;

    const b = data[i + 2] / 255;

    const exg = Math.max(0, Math.min(1, 2 * g - r - b));

    const val = Math.round(exg * 255);

    data[i] = val;

    data[i + 1] = val;

    data[i + 2] = val;

  }

  return imageData;

}
```

A.2 src/services/geminiService.ts — Gemini AI Integration

```
// geminiService.ts — Gemini AI client

import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(

  import.meta.env.VITE_GEMINI_API_KEY

);
```

```
export async function diagnoseLeaf(base64Image: string): Promise<string> {  
  const model = genAI.getGenerativeModel({ model: 'gemini-pro-vision' });  
  const prompt = `You are an expert plant pathologist. Analyze this leaf image and provide:  
    1. Disease name (if any)  
    2. Visible symptoms  
    3. Severity level  
    4. Recommended treatment`,  
  const result = await model.generateContent([prompt, {  
    inlineData: { mimeType: 'image/jpeg', data: base64Image }  
  }]);  
  return result.response.text();  
}
```

A.3 Environment Configuration (.env)

```
# .env — Vite environment variables  
VITE_GEMINI_API_KEY=your_api_key_here
```

A.4 Local Development Commands

```
npm install      # install dependencies  
npm run dev      # start dev server (Vite)  
npm run build    # production build  
npm run preview  # preview production build
```

Department of Computer Science and Engineering

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

The following descriptions outline the key screens of the AgroPulse application as observed during testing.

Screen	Description
Home Screen	Displays the AgroPulse header, Open Field Camera button, and Upload from Gallery option side by side.
Camera Preview	Shows the live browser camera feed inside a rounded preview panel. A capture button is centered below the video.
Image Preview	After capture or upload, the selected leaf image is displayed with the Analyze Now button enabled below.
ExG Processed View	The image is enhanced with the Excess Green filter, highlighting the green leaf regions against the background.
Diagnosis Output	The Gemini AI response is rendered as structured Markdown showing disease name, symptoms, severity, and treatment recommendations.
Error State	If camera permission is denied, a descriptive error message is shown along with a prompt to use the upload option instead.

